

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	40% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	SYED MUHAFIZ A	Reg. No.:	192472282
Section / Batch:		Date:	

Code of Conduct

I SYED MUHAFIZ A (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyze multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

1. Banking Chatbot — CFG Parsing of Ambiguous Queries

A banking support chatbot parses free-text queries with a Context-Free Grammar (CFG) so that structured intents (e.g., "retrieve transaction history") can be mapped onto core-banking API calls. The chatbot currently mis-handles ambiguous phrasing, subject-verb agreement, and long queries. The worked example query is: "Show me the transactions with the card from last month."

1(a) Ambiguity Analysis Using CFG

A minimal CFG sufficient to parse the example query is:

```
S    -> VP
VP   -> V NP NP
NP   -> Pron
NP   -> Det N
NP   -> Det N PP
PP   -> P NP
V    -> "Show"
Pron -> "me"
Det  -> "the" | "last"
N    -> "transactions" | "card" | "month"
P    -> "with" | "from"
```

Grammar 1.1 – Minimal CFG covering the example query

This grammar is deliberately under-constrained (as most hand-written CFGs are) and licenses at least two distinct derivations for the object noun phrase "the transactions with the card from last month", because the prepositional phrase "from last month" can attach at two different points in the tree — the classic PP-attachment ambiguity:

Reading 1 — "from last month" modifies "transactions" (temporal scope of the transaction history)

```
NP
├── Det: the
├── N: transactions
├── PP: with the card
└── PP: from last month
```

Meaning: transactions [that used the card], [restricted to last month]

Reading 2 — "from last month" modifies "card" (recency of the card, not the transactions)

```
NP
├── Det: the
├── N: transactions
└── PP: with
    └── NP
        ├── Det: the
        ├── N: card
        └── PP: from last month
```

Meaning: transactions [using the card that was issued/added last month] – no time filter on the transactions themselves

Both trees are equally well-formed under the given CFG, and a plain (non-probabilistic) CFG parser has no principled way to prefer one over the other — it either returns both (ambiguous parse forest) or arbitrarily returns whichever the parsing algorithm happens to construct first. In a banking context this is

not a cosmetic issue: Reading 1 should filter the transaction query by last month, while Reading 2 would return the customer's entire transaction history with no time filter at all, producing an incorrect and potentially compliance-relevant response (wrong statement period returned to the customer).

1(b) Limitations of CFG for Real-World Conversational Queries

- No mechanism to rank ambiguous parses: as shown above, CFG treats every grammatically licensed derivation as equally valid, so the system cannot automatically prefer the linguistically/contextually more probable Reading 1 over Reading 2.
- No agreement checking: plain CFG rules such as NP → Det N do not encode number/person features, so ungrammatical combinations ("the transaction was showing" vs. "the transactions was showing") are either both accepted or both rejected depending on how liberally the grammar is written — there is no built-in constraint propagation.
- Combinatorial explosion on long queries: each additional prepositional phrase or clause roughly multiplies the number of licensed attachment sites, so a CFG parser's search space (and, for naive parsers, its runtime) grows rapidly with sentence length — a serious concern for verbose or run-on customer queries.
- No semantic/world-knowledge filtering: CFG is purely syntactic, so it cannot rule out a structurally valid but semantically implausible parse (e.g., attaching "from last month" to "me" would be syntactically excluded here, but subtler semantic errors elsewhere in the grammar are not caught).
- Brittleness to conversational disfluency: real chat/voice input contains fillers, typos, incomplete sentences, and code-switching (e.g., "show me txns w/ the card last mnth"), none of which a hand-written CFG's fixed rule set anticipates, causing outright parse failure rather than a degraded-but-usable partial parse.
- Static grammar maintenance cost: extending the grammar to cover every new banking phrasing ("card ending in 4321", "joint account transactions") requires manually adding rules, which does not scale as the chatbot's supported query vocabulary grows.

1(c) Proposed Improvement: PCFG + Feature Structures + Earley Parsing

A production-grade solution replaces the single hand-written CFG with a three-layer pipeline that keeps CFG's interpretable rule structure but adds the statistical, agreement, and efficiency machinery it lacks:

1. Probabilistic CFG (PCFG): attach a learned probability to every production rule (e.g., $P(\text{PP attaches to N "transactions" | head="transactions"})$ vs. $P(\text{PP attaches to N "card"})$), trained on a corpus of historical banking chat logs annotated with correct attachments. At parse time, the parser returns the highest-probability derivation (via the Viterbi/CKY-with-probabilities algorithm) instead of an undifferentiated set of trees — resolving the Reading 1 vs. Reading 2 ambiguity from part (a) using data-driven preference (in banking chat logs, "with the card" + "from last month" attaching jointly to "transactions", i.e., Reading 1, is empirically far more frequent).
2. Feature Structures for agreement and typing: augment each grammar symbol with a feature bundle (e.g., $\text{NP}[\text{NUM}=\text{plural}, \text{ENTITY}=\text{TransactionSet}]$, $\text{N}[\text{NUM}=\text{singular}, \text{ENTITY}=\text{Card}]$, $\text{PP}[\text{TIME-RANGE}=\text{last-month}]$). Parses proceed only when features unify (e.g., a plural subject must unify with a plural-agreeing verb), which simultaneously (i) rejects subject–verb agreement violations and (ii) uses semantic typing ($\text{ENTITY}=\text{Card}$ vs. $\text{ENTITY}=\text{TransactionSet}$) to prefer the attachment that is entity-consistent, giving a principled, symbolic complement to the PCFG's statistical preference.
3. Earley Parsing as the parsing engine: replace the naive recursive-descent/backtracking CFG parser with an Earley chart parser, which handles arbitrary CFGs (including left-recursive rules

common in NP -> NP PP banking-query grammars) in worst-case $O(n^3)$ time and near-linear time for the well-behaved, mildly-ambiguous grammars typical of query parsing. Earley's chart naturally represents all live parses as a shared, compact structure (a parse forest) rather than re-deriving overlapping sub-parses from scratch, which is what makes it tractable for long queries where naive CFG backtracking would blow up combinatorially.

In combination: Earley parsing builds the shared parse forest efficiently; feature-structure unification prunes agreement- and type-inconsistent branches from that forest as it is built; and the PCFG probabilities rank whatever grammatically and semantically valid parses remain, so the system returns one confident, correct interpretation rather than an ambiguous or wrong one.

1(d) Justification of Accuracy and Efficiency Gains

Problem in Original System	Root Cause	How the Proposed Method Fixes It
Incorrect interpretation of ambiguous sentences	Plain CFG has no way to rank equally-valid parses	PCFG assigns learned probabilities so the most likely (data-supported) attachment is automatically selected
Failure to handle subject-verb agreement	CFG symbols carry no number/person information	Feature structures unify NUM/PERSON features across the parse, rejecting disagreement automatically
Inefficient parsing for long queries	Naive/backtracking parsers re-explore overlapping sub-derivations	Earley's chart (dynamic-programming) parsing shares sub-parses, giving cubic worst-case / near-linear typical-case performance

Table 1.1 – Mapping each original limitation to its fix

From a business standpoint, the improved pipeline reduces two costs that matter directly to a bank: incorrect-interpretation risk (a wrong transaction-period response is a customer-trust and potential regulatory issue) and response-latency cost (Earley's shared parse forest avoids the exponential blow-up that would otherwise slow down or time out on long, multi-clause customer queries). Because probabilities and feature constraints are learned/updated from logged interactions rather than hand-coded, the system also improves continuously as more banking-domain query data is collected, unlike a static hand-written CFG that requires manual rule engineering for every new phrasing pattern.

2. Voice Assistant — Top-Down vs. Earley Parsing

A voice assistant (Alexa/Siri-style) currently parses spoken commands with a top-down parser, and suffers from backtracking overhead, poor handling of incomplete/ambiguous input, and difficulty meeting real-time response targets. The example command is: “Book a flight to Delhi with a window seat.”

2(a) Multiple Parse Structures for the Example Command

The ambiguity again centres on prepositional-phrase attachment — this time for “with a window seat”, which can attach at the verb-phrase level or the noun-phrase level:

Reading 1 — “with a window seat” attaches to the VP (a booking-preference modifier of the action “Book”)

```
VP
├ V: Book
├ NP: a flight to Delhi
└ PP: with a window seat      <- modifies the booking action

Meaning: perform the booking, [with the added preference of] a window seat
```

Reading 2 — “with a window seat” attaches to the NP (a property of the flight itself)

```
VP
├ V: Book
└ NP
  ├── NP: a flight to Delhi
  └ PP: with a window seat    <- modifies "a flight to Delhi"

Meaning: book [a flight to Delhi that has a window seat] – describes the flight, not
the act of booking
```

Semantically the two readings converge on a similar real-world outcome (assign a window seat), which is precisely why this class of PP-attachment ambiguity is so common and so easy for a purely syntactic parser to leave unresolved — the grammar alone cannot tell the assistant whether “window seat” is a slot value attached to the booking action or a modifier of the flight entity, yet the two structures could diverge sharply for a longer command (e.g., “Book a flight to Delhi with a stopover in Mumbai”, where VP- vs. NP-attachment of “with a stopover in Mumbai” changes whether the stopover is a property of the route or an instruction about how the booking should be performed).

2(b) Limitations of Top-Down Parsing in a Real-Time Voice Application

- Backtracking cost: a top-down parser predicts a grammar rule and expands it before checking whether the actual input matches; when a prediction fails partway through, the parser must undo the partial derivation and try the next alternative rule, and with attachment ambiguities like the one above this retry cost compounds across every ambiguous point in the sentence.
- No native support for left-recursive rules: naturally-written NP → NP PP style rules (needed to attach chained modifiers such as “a flight to Delhi with a window seat on the 10am departure”) cause a standard top-down parser to recurse infinitely without special grammar transformation, forcing awkward, error-prone grammar rewriting to avoid left recursion.
- Poor handling of incomplete input: voice input arrives incrementally (streaming ASR) and may be interrupted, contain false starts, or be temporarily ambiguous mid-utterance (“Book a flight to...” before the destination is spoken); a top-down parser that has committed to a specific rule

expansion is poorly positioned to gracefully accept a partial utterance or revise cleanly when more audio arrives.

- Repeated backtracking harms real-time responsiveness: because voice assistants must return a response (or at least a partial acknowledgement) within a tight latency budget, every backtrack-and-retry cycle directly eats into the time budget, and worst-case behaviour on ambiguous or noisy ASR-transcribed input can be exponential in sentence length.
- Fragility to ASR noise: speech-to-text errors (“window” misheard as “winter”, dropped articles) frequently produce input that matches no rule expansion the top-down predictor tried, causing an outright parse failure rather than a degraded/approximate parse the user experience could still work with.

2(c) How Earley Parsing Improves Ambiguity and Partial-Input Handling

1. Chart-based dynamic programming instead of prediction-and-backtrack: Earley parsing builds a chart of states across the input, where each state records a grammar rule, how much of it has been matched, and where it started; states are added incrementally as each word arrives, so the algorithm never has to discard and redo work the way a top-down backtracking parser does.
2. Native handling of ambiguity: because the chart keeps all valid partial derivations simultaneously as separate states in the same Earley set, both Reading 1 and Reading 2 for “with a window seat” exist side-by-side in the chart without one blocking the other — the assistant can defer the attachment decision until enough context (or a PCFG-style scoring pass) resolves it, rather than the parser being forced to commit early and backtrack if wrong.
3. Left-recursion support out of the box: Earley’s algorithm handles left-recursive productions (NP → NP PP) directly and correctly, so the grammar for chained flight-booking modifiers does not need to be rewritten into a less natural right-recursive form purely to satisfy the parsing algorithm.
4. Graceful handling of partial/incomplete input: because the chart is built left-to-right and incrementally, the parser’s state after processing “Book a flight to...” already reflects every grammatically consistent continuation; this is directly usable for streaming ASR, where the assistant can begin acting on high-confidence partial structure (e.g., recognising “Book a flight” as the core intent) before the full utterance, including trailing modifiers like the seat preference, has finished arriving.
5. Predictable worst-case complexity: Earley parsing is $O(n^3)$ in the worst case for arbitrary CFGs, $O(n^2)$ for unambiguous grammars, and effectively linear for many real, mildly-ambiguous grammars such as command grammars — all without the potentially exponential blow-up that repeated top-down backtracking can exhibit on ambiguous or noisy input.

2(d) Comparative Performance: Top-Down vs. Earley Parsing

Dimension	Top-Down Parsing	Earley Parsing
Handling of ambiguity	Explores one alternative at a time; must backtrack on failure	Maintains all valid derivations in parallel within the chart
Left recursion	Fails/infinite loop without grammar rewriting	Handled natively, no grammar transformation needed
Partial / incomplete input	Poor — commits to a rule expansion before seeing full input	Well suited — chart updates incrementally as tokens arrive
Worst-case time complexity	Exponential on ambiguous/backtracking-heavy grammars	$O(n^3)$ worst case; near-linear on typical command grammars

Dimension	Top-Down Parsing	Earley Parsing
Robustness to ASR noise	Low — a single mismatch can trigger costly re-derivation	Higher — partial matches remain valid states rather than dead ends
Suitability for real-time voice assistants	Limited beyond small, low-ambiguity command sets	Strong — supports streaming, low-latency, ambiguity-tolerant parsing

Table 2.1 – Top-down vs. Earley parsing for real-time voice-command parsing

For a narrow, hand-curated command grammar with very little ambiguity, top-down parsing's simplicity and low implementation overhead can be acceptable. However, as a voice assistant's supported vocabulary and phrasing variety grows — exactly the trajectory of a commercial Alexa/Siri-class product — the exponential backtracking risk and poor partial-input handling of top-down parsing become a genuine reliability and latency liability, making Earley parsing (or an equivalent chart-parsing / dynamic-programming approach) the more defensible industry choice.

3. Healthcare Report Extraction — Integrated NLP Architecture

A healthcare information-extraction system must convert long, clinically dense sentences into structured records (diagnosis, treatment, follow-up actions) for hospital use. The current CFG-only system fails on the example sentence: “The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.” This sentence combines a relative clause, a coordinated verb phrase, and two competing prepositional/temporal attachments — well beyond what plain CFG reliably resolves, as established in Sections 1 and 2.

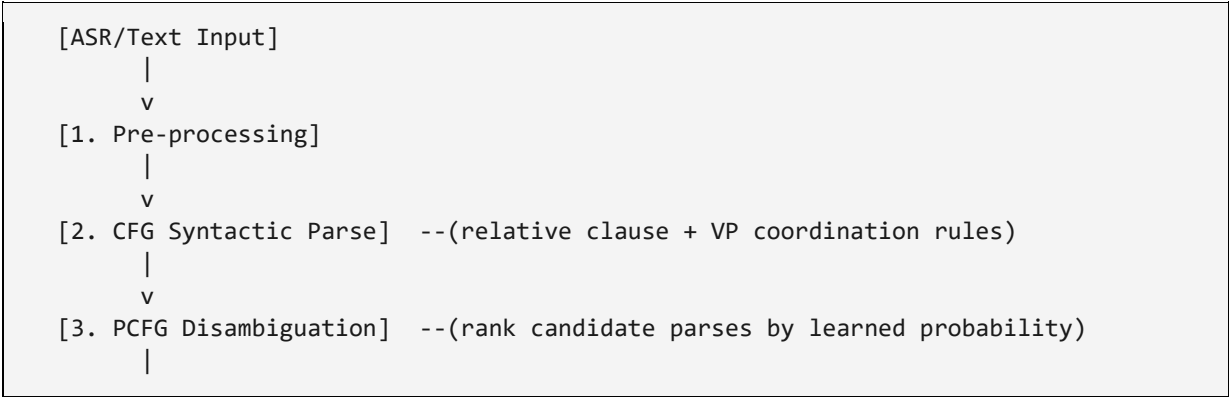
3(a) Integrated NLP Architecture Design

The proposed architecture is a five-layer pipeline in which each layer solves exactly the failure mode a CFG-only system exhibits:

Layer	Component	Purpose
1	Pre-processing	Sentence segmentation, tokenization, clinical abbreviation expansion (e.g., "f/u" -> "follow-up"), spelling normalisation
2	CFG — Syntactic Backbone	Domain-adapted grammar rules (including relative clauses: NP -> NP SBAR, SBAR -> Rel-Pron VP; and VP coordination: VP -> VP Conj VP) provide the base tree structure
3	PCFG — Ambiguity Resolution	Rule probabilities, trained on an annotated clinical treebank, select the most probable attachment for temporal/locative modifiers (e.g., "last week" -> "reviewed", not -> "recommends")
4	Feature Structures — Agreement & Typing	Unify NUM/PERSON (doctor[3sg] agrees with recommends[3sg]), and semantic ENTITY features (Patient, Medication, Location=Chennai, Time=last-week) across the tree
5	Earley / Chart Parser — Efficient Parsing Engine	Cubic-time, ambiguity-tolerant parsing of the full (long, coordinated, relative-clause-bearing) sentence without combinatorial blow-up

Table 3.1 – Five-layer architecture, mapped to the failure mode each layer resolves

On top of this parsing core sit two extraction-specific components: a Sub-Categorization Frame Lexicon for medical verbs (Section 3c) that supplies each verb's expected argument structure (Agent, Theme, Time, Location), and a Semantic Role Labelling / Frame-Filling layer that reads the disambiguated, feature-annotated, sub-categorized parse tree and emits the final structured record (Diagnosis / Action / Location fields).



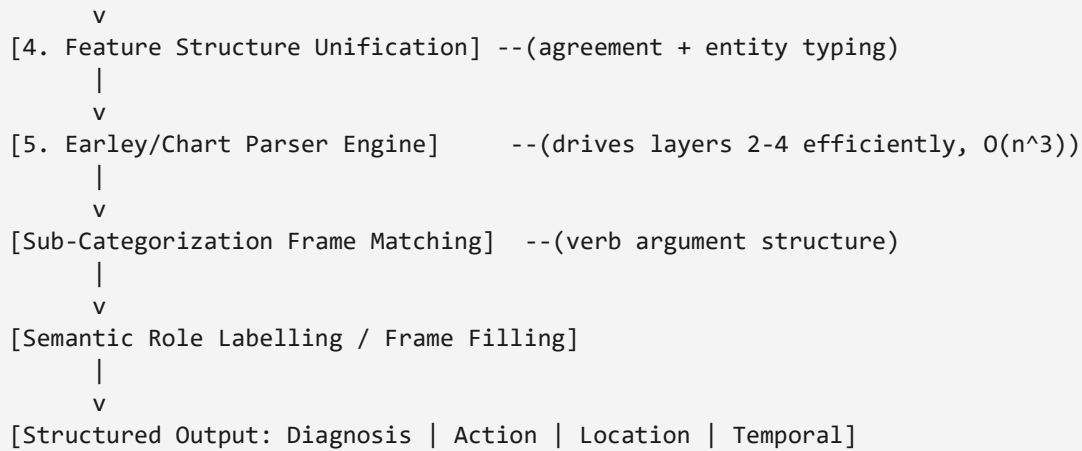


Figure 3.1 – End-to-end architecture pipeline

3(b) Step-by-Step Workflow for the Example Sentence

Input: “The doctor who reviewed the patient last week recommends starting medication and scheduling a follow-up visit in Chennai.”

1. Pre-processing: sentence segmented as a single unit; tokenized into [The, doctor, who, reviewed, the, patient, last, week, recommends, starting, medication, and, scheduling, a, follow-up, visit, in, Chennai]; no abbreviations present, so no expansion needed.
2. CFG syntactic parse: the parser identifies the main clause subject NP “The doctor” modified by a relative clause SBAR “who reviewed the patient last week”, and a coordinated main-clause VP “recommends [VP starting medication] and [VP scheduling a follow-up visit in Chennai]”, correctly nesting the relative clause inside the subject NP rather than treating it as a second main clause.
3. PCFG disambiguation: two attachment decisions are scored — (i) “last week” is resolved to attach to “reviewed” (inside the relative clause) rather than to “recommends” (the main verb), because the learned probabilities favour a temporal modifier attaching to the nearest preceding verb it is compatible with, matching normal clinical-note usage where the review date, not the recommendation date, is the one specified; (ii) “in Chennai” is resolved to attach to “follow-up visit”, not to “medication”, since medication does not take a locative argument while a visit/appointment does.
4. Feature structure unification: agreement is checked and confirmed — “The doctor” [NUM=sg, PERSON=3] unifies with “recommends” [NUM=sg, PERSON=3]; the relative pronoun “who” co-indexes with “The doctor” so the embedded clause’s implicit Agent is correctly bound to the same entity, not misread as a second, unnamed reviewer.
5. Sub-categorization frame matching: “recommends” is matched against its clinical verb frame `recommend(Agent, Theme=[VP-gerund-list])`, correctly taking the coordinated gerund phrases as its Theme argument; “reviewed” is matched against `review(Agent, Theme=Patient, Time)`; “scheduling” is matched against `schedule(Agent, Object=Event, Location)` — supplying the expected argument slots (see Section 3c) that the frame-filling step will populate.
6. Semantic role labelling / frame filling: the disambiguated, feature-checked, sub-categorized tree is converted into filled semantic frames: `Review-Event{Agent=Doctor, Theme=Patient, Time="last week"}`; `Recommend-Event{Agent=Doctor, Theme=[Start-Medication-Event, Schedule-FollowUp-Event]}`; `Schedule-FollowUp-Event{Agent=Doctor(implicit), Object="follow-up visit", Location="Chennai"}`.
7. Structured output: the frames are serialised into the hospital system’s structured record format.

```

{
  "physician": "doctor",
  "review": { "subject": "patient", "time": "last week" },
  "recommended_actions": [
    { "action": "start_medication" },
    { "action": "schedule_follow_up_visit", "location": "Chennai" }
  ]
}

```

Output 3.1 – Final structured record produced from the example sentence

3(c) Sub-Categorization Frames, Real-Time Processing, and Scalability

Incorporating sub-categorization frames for medical verbs

Build a clinical verb-frame lexicon that specifies, for each medically relevant verb, the semantic roles it obligatorily or optionally takes, so the extraction layer knows exactly which slots to look for and validate rather than inferring structure generically:

Verb	Sub-categorization Frame
recommend	recommend(Agent:Clinician, Theme:{Treatment/Action, ...})
review	review(Agent:Clinician, Theme:Patient, [Time])
prescribe	prescribe(Agent:Clinician, Theme:Medication, Recipient:Patient, [Dosage])
schedule	schedule(Agent:Clinician, Object:Event, [Location], [Time])
diagnose	diagnose(Agent:Clinician, Theme:Patient, Result:Condition)

Table 3.2 – Sample sub-categorization frame lexicon for clinical verbs

At parse time, once a verb is identified, its frame acts as a template the sub-categorization matcher tries to fill from the syntactic/semantic-role output; missing obligatory slots (e.g., a “prescribe” event with no identifiable Medication argument) can be flagged for human review rather than silently producing an incomplete record, directly improving output reliability over a generic parser with no verb-specific expectations.

Handling real-time hospital data processing

- Streaming/incremental parsing: run the Earley chart parser incrementally as clinical text is entered (dictation, EHR free-text fields), so partial structure is available immediately rather than waiting for an entire multi-sentence report.
- Micro-service, asynchronous pipeline: decouple pre-processing, parsing, and frame-filling into independently scalable services connected via a message queue (e.g., Kafka), so a burst of admissions/reports does not block real-time throughput for any single stage.
- Caching of grammar/PCFG state: cache parse-chart fragments and PCFG rule probabilities for common clinical sentence templates (a small set of syntactic patterns covers a large share of real clinical narrative), reducing repeated computation.
- Confidence-based fast path vs. review path: high-confidence PCFG/feature-structure outputs are auto-committed to the structured record; low-confidence or frame-incomplete outputs are routed to a clinician-review queue rather than delaying the entire pipeline while a single sentence is resolved.

Improving accuracy and scalability in a hospital environment

1. Domain-adapted training data: train PCFG probabilities and feature-structure typing on annotated clinical corpora (e.g., de-identified discharge summaries, physician notes) rather than general-domain text, since clinical syntax and vocabulary differ substantially from everyday language.
2. Hybrid symbolic + neural extraction: pair the symbolic CFG/PCFG/feature-structure pipeline (which gives interpretable, auditable structure — important for clinical/regulatory accountability) with a neural named-entity recognizer (e.g., a BioBERT-style clinical language model) for robust entity detection, using the neural output as an additional feature input to the sub-categorization matcher rather than replacing the symbolic layer outright.
3. Active learning from clinician feedback: capture corrections made during the human-review path and feed them back into periodic PCFG/feature-structure retraining, so the system's accuracy compounds over time on the specific hospital's documentation style.
4. Horizontal scalability: containerize each pipeline stage so additional parsing capacity can be added elastically during peak admission periods, and keep all processing on-premises or in a compliant private cloud to satisfy hospital data-privacy (e.g., HIPAA-equivalent) requirements without sacrificing throughput.
5. Continuous evaluation against gold-standard records: maintain a held-out set of manually annotated reports to continuously monitor extraction accuracy (diagnosis/action/location field-level precision and recall) as the underlying grammar, probabilities, and neural components evolve, catching regressions before they reach clinical use.

4. Consolidated Conclusion

The three case studies converge on the same underlying engineering pattern: a bare Context-Free Grammar is necessary but not sufficient for real-world conversational and clinical NLP, because it can describe what is grammatically possible but has no mechanism for choosing what is probable (PCFG), no mechanism for enforcing agreement or entity-type consistency (feature structures), and — when implemented with naive top-down or backtracking parsers — no efficient mechanism for handling long, ambiguous, or partially-arriving input (Earley/chart parsing).

Across banking, voice-assistant, and healthcare domains, the same three-part remedy recurs: add probabilities to rank ambiguous derivations, add feature unification to enforce grammatical and semantic consistency, and replace backtracking parsers with a chart-based (Earley) engine that shares sub-parses instead of recomputing them. The healthcare architecture in Section 3 shows how these three techniques compose into a single production pipeline, additionally layered with sub-categorization frames for domain verbs and an operational design (streaming, caching, confidence-gated human review, active learning) that turns the theoretical parsing improvements into a system that is accurate, auditable, and scalable enough for real hospital and enterprise deployment.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q. No. / Criteria Level	Understanding of Industry Problem	Application of Engineering Concepts	Solution Design	Use of Tools	Analysis	Professionalism	Sub. Total	Total %
1								
2								
3								

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____